

■ Lab 11 — Provisioners : Déployer Nginx

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Utilisé un **provisioner** `remote-exec` pour installer Nginx sur une instance EC2
- Utilisé un **provisioner** `local-exec` pour exécuter une commande en local
- Compris la différence entre **Creation Time** et **Deletion Time** provisioners
- Configuré une **key pair** SSH pour accéder à l'instance

■ Étape 1 — Créer la Key Pair SSH

```
cd labs/lab11-provisioners

# Générer une paire de clés SSH
ssh-keygen -t rsa -b 2048 -f lab11-key -N ""
```

■ Étape 2 — Créer les fichiers

`backend.tf`

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■ ■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

`provider.tf`

```
provider "aws" {
  region = var.region
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prenom"
  type        = string
}

variable "region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-1"
}
```

`locals.tf`

```
locals {
  prefix = "lab11-${var.username}"

  common_tags = {
    Lab       = "lab11"
    Username  = var.username
    ManagedBy = "Terraform"
  }
}
```

`main.tf`

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

# Key pair pour accéder à l'instance
resource "aws_key_pair" "lab11_key" {
  key_name     = "${local.prefix}-key"
  public_key   = file("${path.module}/lab11-key.pub")

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-key"
  })
}

# Security Group avec SSH et HTTP
resource "aws_security_group" "lab11_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group Lab 11 - ${var.username}"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "SSH"
  }
}
```

```
}

ingress {
  from_port = 80
  to_port   = 80
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  description = "HTTP"
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

tags = merge(local.common_tags, {
  Name = "${local.prefix}-sg"
})
}

resource "aws_instance" "lab11_instance" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = "t2.micro"
  key_name              = aws_key_pair.lab11_key.key_name
  vpc_security_group_ids = [aws_security_group.lab11_sg.id]
  associate_public_ip_address = true

  tags = merge(local.common_tags, {
```

```

    Name = "${local.prefix}-nginx"
  })

# Provisioner local-exec : s'exécute sur la machine locale après création
provisioner "local-exec" {
  command = "echo 'Instance créée : ${self.public_ip}' >> lab11-instances.txt"
}

# Provisioner remote-exec : s'exécute sur l'instance distante
provisioner "remote-exec" {
  inline = [
    "sudo dnf update -y",
    "sudo dnf install -y nginx",
    "sudo systemctl start nginx",
    "sudo systemctl enable nginx",
    "echo '<h1>Lab 11 - ${var.username}</h1>' | sudo tee /usr/share/nginx/html/index.html"
  ]

  connection {
    type      = "ssh"
    user      = "ec2-user"
    private_key = file("${path.module}/lab11-key")
    host      = self.public_ip
  }
}
}

```

■ **local-exec** s'exécute sur la machine qui lance Terraform (votre Codespace/CloudShell).

remote-exec s'exécute sur l'instance EC2 distante via SSH.

Par défaut, les provisioners s'exécutent à la **création** de la ressource.

outputs.tf

```

output "instance_public_ip" {
  description = "IP publique de l'instance Nginx"
  value      = aws_instance.lab11_instance.public_ip
}

output "nginx_url" {
  description = "URL pour accéder à Nginx"
  value      = "http://${aws_instance.lab11_instance.public_ip}"
}

```

■ Étape 3 — Déployer et tester Nginx

```
terraform init
terraform apply -var="username=<votre-prenom>"
```

■ Le déploiement prend 2-3 minutes — Nginx doit être installé sur l'instance.

Testez Nginx :

```
NGINX_IP=$(terraform output -raw instance_public_ip)
curl http://$NGINX_IP
```

Résultat attendu :

```
<h1>Lab 11 - <votre-prenom></h1>
```

■ Étape 4 — Vérifier le provisioner local-exec

```
cat lab11-instances.txt
```

Résultat attendu :

```
Instance créée : <IP_PUBLIQUE>
```

■ Étape 5 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
rm -f lab11-key lab11-key.pub lab11-instances.txt
```

■ Validation du Lab

#	Critère	Validé
1	Key pair SSH créée avec `ssh-keygen`	■
2	`terraform apply` installe Nginx sur l'instance	■
3	`curl http://` retourne la page HTML Lab 11	■

#	Critère	Validé
4	`lab11-instances.txt` contient l'IP de l'instance	■
5	`terraform destroy` supprime toutes les ressources	■

■ ■ Résolution des problèmes courants

Problème	Cause	Solution
`Connection refused`	Instance pas encore prête	Attendre 30s et relancer
`Permission denied`	Mauvaise clé SSH	Vérifier que `lab11-key` est dans le bon dossier
`Nginx not reachable`	Security Group mal configuré	Vérifier le port 80 dans le SG

■ ****Fin du Lab 11 — Nginx est déployé automatiquement avec Terraform !****

*Le Lab 12 introduit les ****modules**** pour organiser et réutiliser votre code Terraform.*